Mandate Language Specifications

Specification v1.0.0

Table of Contents

1. Preamble

Mandate is a functional, statically typed programming language created to be embedded in JavaScript applications. This language can be run from source. Mandate is designed be used within another program.

Mandate is free and open source software. It is licensed under the terms of the LGPL 2.0 or greater license; implementation source code can be found at https://github.com/Mandate-lang/Mandate-specs.

This specification is licensed under CC-BY-SA 4.0, and any code snippets are licensed under the Unlicense. For more licence information, see Section 3 of this document.

1.1 Why Mandate?

Mandate is named because of the legalistic inspiration for its syntax. “Mandate” was chosen as it was not a current name for a language, nor was “Mandate-lang” a current npm package.

2. The Source Language

This section describes the language used as source code.

2.1 Types

2.1.1 The Integer

The integer is the data type for storing whole numbers.

```
-52
100
123456789
77971101009711610110
```

The integer has no max size limit except the size of memory.

2.1.2 The Float

The float is the data type for storing real numbers. Mandate floats follow the IEEE 754 standard for floating point mathematics.

```
-50.09
3.14
34.482
```

Floats MUST have a leading zero for numbers between -1 and 1, and MUST have a zero following the decimal point when representing an integer as a float.\

```
0.123
123.0
```

2.1.3 The Boolean

The boolean is the data type for storing True and False values.

```
true
false
```

2.1.4 The String

The string is the data type for storing character data. String/character data is inclosed between double quotes (“).

```
“Hello, World”
```

Special characters such as tab, newline, and double quotes can be included using escape sequences.

2.1.4.1 Escape Sequences

Escape sequences can be used to add special characters in strings. Escape sequences are prefixed by the reverse solidus, Unicode Point U+005C (\).

Value	Escape Sequence
New Line	`\n`
Horizontal Tab	`\t`
Backspace	`\b`
Carriage Return	`\r`
Vertical Tab	`\v`
Null Character	`\0`
Form Feed	`\f`
Backslash	`\\`
Double Quote	`\”`
Unicode Point	`\uhhhhhh`

The Unicode Point escape code uses 6 hexadecimal digits to represent any character. All six digits are REQUIRED. Leading zeros can be used to fill the extra fields for shorter code points.

```
\u00042
\u00FF54
```

2.1.5 The Array

The array is a data type for storing multiple of a single data type. Arrays can only store one data type. The array starts and ends with square brackets ([]), and values are separated by commas (,). Arrays have zero-based indexing, and are not mutable.

```
[0, 2, 9]
[“This”, “is” “an “array”]
[[1, 2], [3, 4]]
```

2.2 Identifiers

2.2.1 Names

Names are used to identify data stored in variables. Names can be created with the following characters:

- Uppercase and lower case letter. (a-z and A-Z)
- Underscores (_)
- Digits (0-9). A Name cannot start with a digit.

```
name
num0
_var
```

2.2.2 Keywords

Keywords are tokens reserved by Mandate. Keywords cannot be used for variable names

integer	float	boolean	string	array	using		

2.3 Expressions

2.3.1 Mathematical Expressions

Mandate supports classical mathematical operators for mathematical expressions. When a mathematical expression is used with a float and an integer, or a float and a float, the result will be a float. In any other case the result will be an integer. Operations involving floats follow the IEEE 754 standard.

2.3.1.1 Addition

Addition is done using the plus symbol, Unicode Point U+002B (+).

```
5+4
7.0+11.5
```

2.3.1.2 Subtraction

Subtraction is done using the hyphen-minus symbol, Unicode Point U+002D (-).

```
5-4
7.0-11.5
```

2.3.1.3 Multiplication

Multiplication is done using the asterisk symbol, Unicode Point U+002A (*).

```
5*4
7.0*11.5
```

2.3.1.4 Division

Division is done using the solidus symbol, Unicode Point U+002F (/).

```
5/4
7.0/11.5
```

When dividing two integers, the quotient is truncated to the nearest whole number. The quotient is NOT rounded. This is known as integer division.

2.3.1.4 Remainder

Remainder is done using the percent symbol, Unicode Point U+0025 (%).

```
5%4
7.0%11.5
```

When taking the remainder using two integers, the remainder is truncated to the nearest whole number. The remainder is NOT rounded. This is known as integer remainder.

2.3.1.5 Division/Remainder of Zero

When performing integer division or integer remainder where the divisor is zero, a DivideByZeroError is thrown. When the divisor is zero in cases involving floats, the quotient or remainder follows the IEEE 754 standard. Dividing/taking the remainder by zero results in positive or negative infinity, and 0 divided/remainder by 0 results NaN, and infinity divided/remainder of infinity results in NaN.

2.3.2 Comparison Expressions

Comparison expressions always evaluate to boolean values.

2.3.2.1 Greater Than

Checking if a numerical value (integer or float) is greater than another numerical value is done using the greater-than sign, Unicode Point U+003E (>).

```
5>4
7.0>11.5
```

Only numerical values (integers and floats) can be compared using greater than.

2.3.2.2 Less Than

Checking if a numerical value (integer or float) is less than another numerical value is done using the less-than sign, Unicode Point U+003C (<).

```
5<4
7.0<11.5
```

Only numerical values (integers and floats) can be compared using less than.

2.3.2.3 Equal To

Checking if a value is equal to another value is done using two equals signs, two Unicode U+003D Points (==).

```
5 == 4
“h” == 3
true == “l”
```

Comparing values of two different types using equals to always results to false.

2.3.2.4 Not Equal To

Checking if a value is not equal to another value is done using an exclamation point and equals signs, Unicode Points U+0021 and U+003D (!=).

```
5 != 4
“h” != 3
true != “l”
```

Comparing values of two different types using not equals to always results to true.

2.3.3 Boolean Expressions

2.3.3.1 And

Checking if two boolean values are both true can be done with the “and” operator (and).

```
true and false
```

The “and” expression only returns true if both operands are true. The “and” expression short circuits; if the left most operand is false, the right operand will not be evaluated. The “and” operator always returns the leftmost operand.

2.3.3.2 Or

Checking if at least one of two boolean values is true can be done with the “or” operator (or).

```
true or false
```

The “or” expression evaluates to true if one or two of the operands are true. The “or” expression short circuits; if the left most operand is true, the right operand will not be evaluated. The “or” expression returns the leftmost operand if the leftmost operand is true, or if both operands are false. If the left operand is false and the right operand is true, the right operand is returned.

2.3.3.3 Not

The “not” expression returns the opposite value of the boolean value provided.

```
not false
```

Using not on a true value returns false, using not on a false value returns true.

2.3.4 Binding Expression

The binding expression is used to bind a value to a variable. Once a variable is bound, it cannot be rebound or be modified. It returns the value on the right side of the expression (VALUE in the syntax below).

```
VARIABLE_NAME refers to a variable of type TYPE with value VALUE.
```

```
new_var refers to a variable of type integer with value 34.

var_2 refers to a variable of type String of value “Hello”.
```

2.3.5 If/Else Expressions

2.3.5.1 Basic If/Else Expressions

If expressions return a value based on a performed comparison expression. If the comparison evaluates to “true” the expression after “then” is evaluated, otherwise the expression after “else” is evaluated.

```
If COMPARISON then EXPRESSION else EXPRESSION.
```

```
If x < 7 then 8 else 4 + 3.
```

The “else” portion is required. Both expressions must evaluate to the same data type.

2.3.5.2 Nested If/Else Expressions

With nested if expressions, else expressions are attached to the innermost if expression. Examples are shown below; grouped if /else expressions are shown through like formatting

```
If COMPARISON then If COMPARISON then EXPRESSION else EXPRESSION
else EXPRESSION.
```

```
If COMPARISON then If COMPARISON then EXPRESSION else if
COMPARISON then EXPRESSION else EXPRESSION else EXPRESSION.
```

```
If COMPARISON then If COMPARISON then if COMPARISON then
EXPRESSION else EXPRESSION else EXPRESSION else EXPRESSION.
```

2.3.6 Grouping

The left and right parenthesis (U+0028 and U+0029) are used to group expressions, overriding order of operations.

```
4 * (2 + 3)
```

2.3.7 Order of Operations

Precedence	Operator	Associativity
1 (Highest)	( )	N / A
2	if/else	Right
3	not	Right
4	* / %	Left
5	+ -	Left
6	< > <= >=	Left
7	== !=	Left

8	`and`	Left
9	`or`	Left
10 (Lowest)	`binding`	Right

2.4 Functions

Functions are the basis of a Mandate program. Functions begin with an integer followed by a period, (such as `0.`), then a name in parentheses, and are made up of expressions. Functions return the value of the last expression in the function. The return type of a function is determined by the last expression in function.

```
1. (builder_func) var_one refers to a variable of type integer
with value 34. var_two refers to a variable of type integer with
value 12.
```

All functions in Mandate are assumed to be pure functions, meaning that they will always have the same output for a given input, and cause no side effects. Because of this, the inputs and the following output are cached when a function is run so the calculations do no have to be completed multiple times. If a function is not pure for any reason, its inputs and outputs are not cached.

2.4.1 Annotations

Annotations are tags that can be placed at the start of a function to specify details about the function. Annotations are not expressions, and so do not return a value. Annotations must come before expressions in a function.

2.4.1.1 Return Type Annotation

The return type can be set to a specific type by including a type setting phrase at the beginning:

```
A value of type TYPE must be returned.
```

2.4.1.2 Parameters Annotation

Parameters can be added to a function with a parameter annotation. If there is no parameter annotation, then the function will have no parameters. The parameter annotation consists of the keyword “using” followed by the types and names of the parameters.

```
Using integer a, string b, array c.
```

2.4.1.3 Impure Annotation

One way to make a function non pure is to add the impure annotation:

```
This is impure.
```

2.4.2 Tail Call Optimisation

A tail call is when a recursive call is the returned expression. The tail call annotation can be used to enforce tail call optimisation:

```
This is optimised.
```

If no optimisation can be performed, an error will be thrown.

2.4.2 Calling Functions

A function calling expression the function id or name (not both) then square brackets containing comma separated arguments.

```
1.[1, 2, var_3]
```

```
new_func.[1, 2, var_3]
```

2.4.3 Nested Functions

Nested functions are shown by their id having sub numbers (i.e. function 1.1 is nested in function 1.). When having multiple nested functions, only the rightmost number in the id is incremented.

2.5 Program Organisation

The basic unit of organisation above functions is the file. A mandate source file has the ".mandate" extension.

2.5.1 File Organisation

Each mandate source file with a "Section" followed by an integer and name at the top:

```
Section 1: Rules
```

The section name follows the same rules as for variable names. The section name should be the same as the name of the mandate source file (i.e. the section "Rules" above should be in a file named "Rules.mandate").

The header is followed by all the program's functions. The function numbers should increase by one through the file

2.5.2 The Main Function

When a mandate program is run, the function numbered "`0.`" is the entry point, and so will be run automatically. The function numbered "`0.`" may have a single String array parameter for

arguments, but a parameter is not required. If there is no function numbered “`0.`”, the program will exit immediately.

2.6 Formal Grammar

Formal grammar is written in EBNF.

```
program = “Section” , positive integer , identifier ,
          { function } ;
function = integer , “ . ” , { integer , “.” } , “(“ ,
           identifier , “)” , { expression } ;
expression = [ “(“ ] , binding expression | if expression
           | math expression | boolean expression
           | comparison expression , [ “)” ] ;
binding expression = identifier , “refers to a variable of type” ,
                     type , “with value” , expression | value ;
if expression = “if” , boolean expression , “then” expression ,
                “else” , expression ;
boolean expression = ( value , “and” | “or” , value )
                   | ( “not” , value ) ;
comparison expression = ( number , “<=” | “>=” , number )
                      | ( value , “==” , “!=” , value ) ;
math expression = number , “+” | “-” | “/” | “*” | “%” ,
                  number ;
value = identifier | number | boolean ;
type = “integer” | “float” | “boolean” | “string” | “array” ;
boolean = “true” | “false” ;
number = integer | float
float = integer  , “.” , positive integer ;
integer = positive integer | “-” , positive integer ;
positive integer = digit , { digit}
identifier = letter , { letter , digit , “_” } ;
digit = “0” | “1” | “2” | “3” | “4” | “5” | “6” | “7” | “8”
      | “9” ;
letter = “A” | “B” | “C” | “D” | “E” | “F” | “G” | “H” | “I” | “J”
       | “L” | “M” | “N” | “O” | “P” | “Q” | “R” | “S” | “T” | “U”
       | “V” | “W” | “X” | “Y” | “Z” | “a” | “b” | “c” | “d” | “e”
       | “f” | “g” | “h” | “I” | “j” | “k” | “l” | “m” | “n” | “o”
       | “p” | “q” | “r” | “s” | “t” | “u” | “v” | “w” | “x” | “y”
       | “z” ;
```

3. Licence

The text of this document is licensed under the CC-BY-SA 4.0 licence:

```
Mandate Specs  © 2026 by greywolfcode is licensed under CC BY-SA 4.0. To
view a copy of this license, visit https://creativecommons.org/licenses/by-
sa/4.0/
```

Any licensable code snippets in this specification are under the Unlicense:

```
This is free and unencumbered software released into the public domain.

Anyone is free to copy, modify, publish, use, compile, sell, or
distribute this software, either in source code form or as a compiled
binary, for any purpose, commercial or non-commercial, and by any
means.

In jurisdictions that recognize copyright laws, the author or authors
of this software dedicate any and all copyright interest in the
software to the public domain. We make this dedication for the benefit
of the public at large and to the detriment of our heirs and
successors. We intend this dedication to be an overt act of
relinquishment in perpetuity of all present and future rights to this
software under copyright law.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS BE LIABLE FOR ANY CLAIM, DAMAGES OR
OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE,
ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR
OTHER DEALINGS IN THE SOFTWARE.
```

```
For more information, please refer to <https://unlicense.org/>
```